Aya Burpee, Brinda Moudgalya
EE/CSE 371
October 4, 2025
Lab2 Report

Design Procedure

In this lab, we worked with memory blocks. For tasks 1 and 3, we used pre-built Quartus IP blocks, and for task 2, we built our own using flipflop logic and memory arrays. The 7-segment displays show the values of the read/write addresses, as well as the data being written to or read from memory. The memory block is a 32x3 RAM module.
Data to be written to memory must be specified by DE1-SoC SW[3:1] and the address to write to is specified by SW[8:4]. SW[0] toggles write enable is on or off, and SW[9] toggles between tasks 2 and 3 (using our implementation of a single-port RAM or the built-in 2-port RAM).

Task #1 – Single-Port RAM Quartus Module
Task #1 was completed by simply installing the proper single-port RAM block from Quartus and instantiating it in the top level DE1_SoC module. The testbenches for Task #1 are shown in the results section.

Task #2 – Single-Port RAM SystemVerilog Module
The output of task #2 was similar to task #1, except the memory module was implemented using SystemVerilog memory blocks and flip flop logic. The unique element of task #2 is that the clock used was KEY[0] rather than CLOCK_50. The function of task #2 is similar to task #1, in that it implements a single-port memory module (accepts one address port) and depending on whether wren (write enable) is ON or OFF, it either writes the data_in input to the address or simply reads out the value stored in that memory cell.

Task #3 – Double-Port RAM Quartus Module with SystemVerilog Implementation Task #3 was the most involved task. For task #3, we implemented a dual port memory module, where the read and write addresses could be different, which was a unique element from the other two tasks. Another interesting part of the assignment for task #3 was that we needed to keep the functionality of task #2. This was a heavy task, and it forced us to implement many different modules and conditional statements to ensure that we were properly performing either task #2 or #3 based on which state SW[9] was in. Since task #2 implemented a single-port RAM and task #3 implemented a dual port RAM, this also meant we needed to figure out how to treat our inputs (we couldn't use the same logic variables, we needed to really design the logic thoroughly). We used a conditional to check the state of SW[9] and set the value of r_address to

either w_address or the counter, implementing either task #3 or task #2, respectively. By thus considering all aspects of the specifications, we were able to adequately design our modules and instantiate everything within our top-level module. Figure 1 shows the top-level block diagram for the full system.

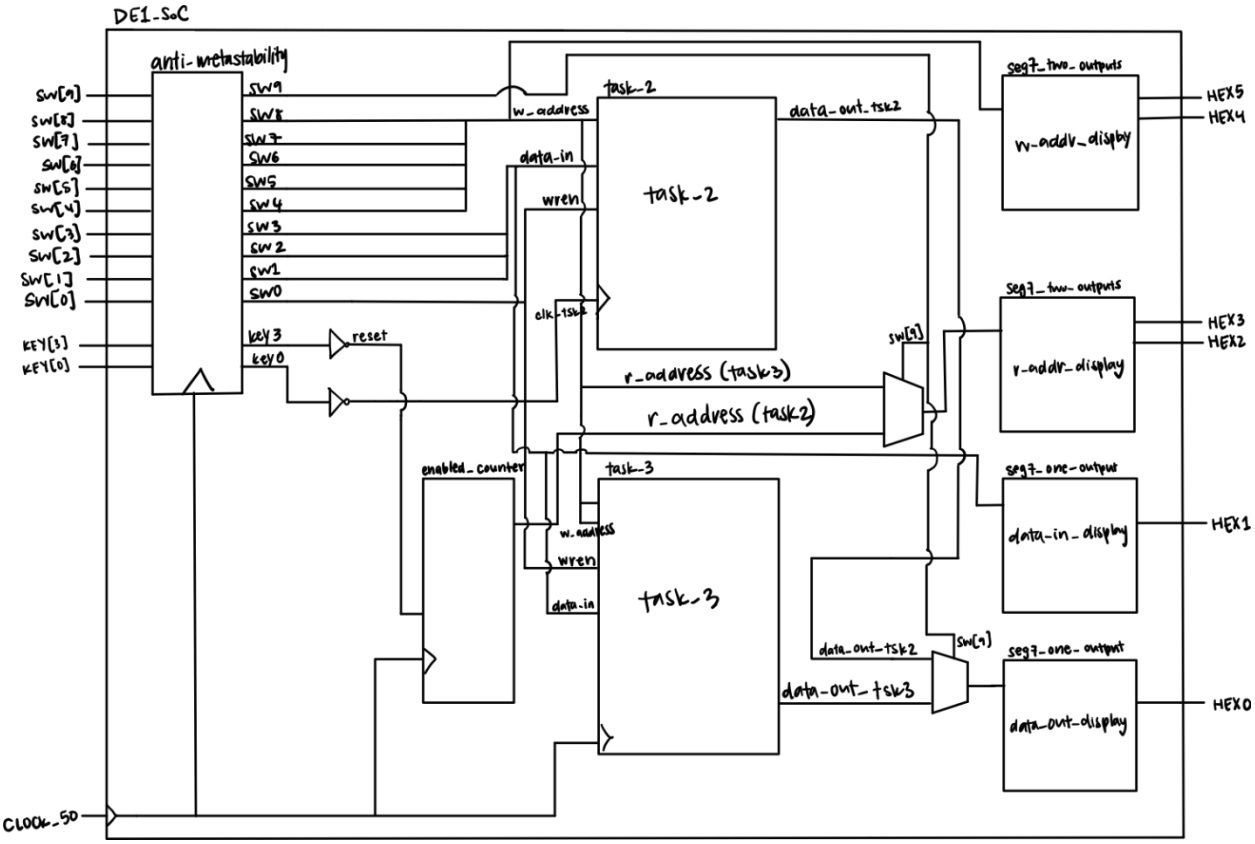


Figure 1. Block diagram of top-level module (DE1_SoC)

Results

The final system was implemented as task #3 and includes a combination of the functionalities of the single-port and dual-port RAM modules. This meant designing a way to toggle between the functionalities of the two RAM modules, since changing the data written to an address of one memory block should not change the data stored in the other. When the user turns on SW[0], they are able to write to a specified address. When SW[0] is off, so is write enable and any data passed through the data_in port is ignored by the memory block. When the user toggles SW[9], they are toggling between the single-port module (~SW[9]) and the dual-port module (SW[9]).

The system was initially testbenched (module by module, then top-level) to ensure functionality of the system. Then the firmware was flashed onto the virtual (LabsLand) DE1_SoC board, and

the expected results were observed. The testbenches are shown in the figures below, and explained.

Task1:

```
# ----------------- write enable ON ------------------
# ----------------- write in all zeros ------------------
# Value of i:           0. Value of address:  0, 00000. Value of data_in: 0, 000. Value of data_out: x, xxx.
# Value of i:           1. Value of address:  1, 00001. Value of data_in: 0, 000. Value of data_out: 0, 000.
# Value of i:           2. Value of address:  2, 00010. Value of data_in: 0, 000. Value of data_out: 0, 000.
# Value of i:           3. Value of address:  3, 00011. Value of data_in: 0, 000. Value of data_out: 0, 000.
# Value of i:           4. Value of address:  4, 00100. Value of data_in: 0, 000. Value of data_out: 0, 000.
# Value of i:           5. Value of address:  5, 00101. Value of data_in: 0, 000. Value of data_out: 0, 000.
# Value of i:           6. Value of address:  6, 00110. Value of data_in: 0, 000. Value of data_out: 0, 000.
# Value of i:           7. Value of address:  7, 00111. Value of data_in: 0, 000. Value of data_out: 0, 000.
# Value of i:           8. Value of address:  8, 01000. Value of data_in: 0, 000. Value of data_out: 0, 000.
# Value of i:           9. Value of address:  9, 01001. Value of data_in: 0, 000. Value of data_out: 0, 000.
# Value of i:          10. Value of address: 10, 01010. Value of data_in: 0, 000. Value of data_out: 0, 000.
# Value of i:          11. Value of address: 11, 01011. Value of data_in: 0, 000. Value of data_out: 0, 000.
# Value of i:          12. Value of address: 12, 01100. Value of data_in: 0, 000. Value of data_out: 0, 000.
# Value of i:          13. Value of address: 13, 01101. Value of data_in: 0, 000. Value of data_out: 0, 000.
# Value of i:          14. Value of address: 14, 01110. Value of data_in: 0, 000. Value of data_out: 0, 000.
# Value of i:          15. Value of address: 15, 01111. Value of data_in: 0, 000. Value of data_out: 0, 000.
# Value of i:          16. Value of address: 16, 10000. Value of data_in: 0, 000. Value of data_out: 0, 000.
# Value of i:          17. Value of address: 17, 10001. Value of data_in: 0, 000. Value of data_out: 0, 000.
# Value of i:          18. Value of address: 18, 10010. Value of data_in: 0, 000. Value of data_out: 0, 000.
# Value of i:          19. Value of address: 19, 10011. Value of data_in: 0, 000. Value of data_out: 0, 000.
# Value of i:          20. Value of address: 20, 10100. Value of data_in: 0, 000. Value of data_out: 0, 000.
# Value of i:          21. Value of address: 21, 10101. Value of data_in: 0, 000. Value of data_out: 0, 000.
# Value of i:          22. Value of address: 22, 10110. Value of data_in: 0, 000. Value of data_out: 0, 000.
# Value of i:          23. Value of address: 23, 10111. Value of data_in: 0, 000. Value of data_out: 0, 000.
# Value of i:          24. Value of address: 24, 11000. Value of data_in: 0, 000. Value of data_out: 0, 000.
# Value of i:          25. Value of address: 25, 11001. Value of data_in: 0, 000. Value of data_out: 0, 000.
# Value of i:          26. Value of address: 26, 11010. Value of data_in: 0, 000. Value of data_out: 0, 000.
# Value of i:          27. Value of address: 27, 11011. Value of data_in: 0, 000. Value of data_out: 0, 000.
# Value of i:          28. Value of address: 28, 11100. Value of data_in: 0, 000. Value of data_out: 0, 000.
# Value of i:          29. Value of address: 29, 11101. Value of data_in: 0, 000. Value of data_out: 0, 000.
# Value of i:          30. Value of address: 30, 11110. Value of data_in: 0, 000. Value of data_out: 0, 000.
# Value of i:          31. Value of address: 31, 11111. Value of data_in: 0, 000. Value of data_out: 0, 000.
# ----------------- write in numbers 0-7 ------------------
# Value of i:           0. Value of address:  0, 00000. Value of data_in: 0, 000. Value of data_out: 0, 000.
# Value of i:           1. Value of address:  1, 00001. Value of data_in: 1, 001. Value of data_out: 0, 000.
# Value of i:           2. Value of address:  2, 00010. Value of data_in: 2, 010. Value of data_out: 1, 001.
# Value of i:           3. Value of address:  3, 00011. Value of data_in: 3, 011. Value of data_out: 2, 010.
# Value of i:           4. Value of address:  4, 00100. Value of data_in: 4, 100. Value of data_out: 3, 011.
# Value of i:           5. Value of address:  5, 00101. Value of data_in: 5, 101. Value of data_out: 4, 100.
# Value of i:           6. Value of address:  6, 00110. Value of data_in: 6, 110. Value of data_out: 5, 101.
# Value of i:           7. Value of address:  7, 00111. Value of data_in: 7, 111. Value of data_out: 6, 110.
# Value of i:           8. Value of address:  8, 01000. Value of data_in: 0, 000. Value of data_out: 7, 111.
# Value of i:           9. Value of address:  9, 01001. Value of data_in: 1, 001. Value of data_out: 0, 000.
# Value of i:          10. Value of address: 10, 01010. Value of data_in: 2, 010. Value of data_out: 1, 001.
# ----------------- write enable OFF ------------------
# Value of i:           0. Value of address:  0, 00000. Value of data_in: 0, 000. Value of data_out: 7, 111.
# Value of i:           1. Value of address:  1, 00001. Value of data_in: 0, 000. Value of data_out: 0, 000.
# Value of i:           2. Value of address:  2, 00010. Value of data_in: 0, 000. Value of data_out: 1, 001.
# Value of i:           3. Value of address:  3, 00011. Value of data_in: 0, 000. Value of data_out: 2, 010.
# Value of i:           4. Value of address:  4, 00100. Value of data_in: 0, 000. Value of data_out: 3, 011.
# Value of i:           5. Value of address:  5, 00101. Value of data_in: 0, 000. Value of data_out: 4, 100.
# Value of i:           6. Value of address:  6, 00110. Value of data_in: 0, 000. Value of data_out: 5, 101.
# Value of i:           7. Value of address:  7, 00111. Value of data_in: 0, 000. Value of data_out: 6, 110.
# Value of i:           8. Value of address:  8, 01000. Value of data_in: 0, 000. Value of data_out: 7, 111.
# Value of i:           9. Value of address:  9, 01001. Value of data_in: 0, 000. Value of data_out: 0, 000.
# Value of i:          10. Value of address: 10, 01010. Value of data_in: 0, 000. Value of data_out: 1, 001.
# Value of i:          11. Value of address: 11, 01011. Value of data_in: 0, 000. Value of data_out: 2, 010.
# Value of i:          12. Value of address: 12, 01100. Value of data_in: 0, 000. Value of data_out: 3, 011.
```

Figure 2. Transcript for task_1 module

Task #1 implemented the Quartus IP single-port memory module. The outputs of the testbench are given in Figure 2. This module was tested individually, as it does not require any other modules. Additionally, since the task was simply to implement a pre-built memory block, only the $display statements are given below, as they more descriptively show the behavior of the module. As can be seen, while write enable is on, we first write all 0s to each memory address. Then, keeping write enable on, we write a 3-bit version of the memory address (the least significant three bits) to each address. When reading out the stored value, data_out is outputted as exactly what was inputted, with one clock cycle delay. When we turn write enable off, we try writing all 0s to the memory addresses, but since write enable is off, we cannot modify the data stored.

Task2:

```
# ----------------- write enable ON -------------------
# ----------------- write in all zeros -------------------
# Value of i:           0. Value of address:  0, 00000. Value of data_in: 0, 000. Value of data_out: x, xxx.
# Value of i:           1. Value of address:  1, 00001. Value of data_in: 0, 000. Value of data_out: 0, 000.
# Value of i:           2. Value of address:  2, 00010. Value of data_in: 0, 000. Value of data_out: 0, 000.
# Value of i:           3. Value of address:  3, 00011. Value of data_in: 0, 000. Value of data_out: 0, 000.
# Value of i:           4. Value of address:  4, 00100. Value of data_in: 0, 000. Value of data_out: 0, 000.
# Value of i:           5. Value of address:  5, 00101. Value of data_in: 0, 000. Value of data_out: 0, 000.
# Value of i:           6. Value of address:  6, 00110. Value of data_in: 0, 000. Value of data_out: 0, 000.
# Value of i:           7. Value of address:  7, 00111. Value of data_in: 0, 000. Value of data_out: 0, 000.
# Value of i:           8. Value of address:  8, 01000. Value of data_in: 0, 000. Value of data_out: 0, 000.
# Value of i:           9. Value of address:  9, 01001. Value of data_in: 0, 000. Value of data_out: 0, 000.
# Value of i:          10. Value of address: 10, 01010. Value of data_in: 0, 000. Value of data_out: 0, 000.
# Value of i:          11. Value of address: 11, 01011. Value of data_in: 0, 000. Value of data_out: 0, 000.
# Value of i:          12. Value of address: 12, 01100. Value of data_in: 0, 000. Value of data_out: 0, 000.
# Value of i:          13. Value of address: 13, 01101. Value of data_in: 0, 000. Value of data_out: 0, 000.
# Value of i:          14. Value of address: 14, 01110. Value of data_in: 0, 000. Value of data_out: 0, 000.
# Value of i:          15. Value of address: 15, 01111. Value of data_in: 0, 000. Value of data_out: 0, 000.
# Value of i:          16. Value of address: 16, 10000. Value of data_in: 0, 000. Value of data_out: 0, 000.
# Value of i:          17. Value of address: 17, 10001. Value of data_in: 0, 000. Value of data_out: 0, 000.
# Value of i:          18. Value of address: 18, 10010. Value of data_in: 0, 000. Value of data_out: 0, 000.
# Value of i:          19. Value of address: 19, 10011. Value of data_in: 0, 000. Value of data_out: 0, 000.
# Value of i:          20. Value of address: 20, 10100. Value of data_in: 0, 000. Value of data_out: 0, 000.
# Value of i:          21. Value of address: 21, 10101. Value of data_in: 0, 000. Value of data_out: 0, 000.
# Value of i:          22. Value of address: 22, 10110. Value of data_in: 0, 000. Value of data_out: 0, 000.
# Value of i:          23. Value of address: 23, 10111. Value of data_in: 0, 000. Value of data_out: 0, 000.
# Value of i:          24. Value of address: 24, 11000. Value of data_in: 0, 000. Value of data_out: 0, 000.
# Value of i:          25. Value of address: 25, 11001. Value of data_in: 0, 000. Value of data_out: 0, 000.
# Value of i:          26. Value of address: 26, 11010. Value of data_in: 0, 000. Value of data_out: 0, 000.
# Value of i:          27. Value of address: 27, 11011. Value of data_in: 0, 000. Value of data_out: 0, 000.
# Value of i:          28. Value of address: 28, 11100. Value of data_in: 0, 000. Value of data_out: 0, 000.
# Value of i:          29. Value of address: 29, 11101. Value of data_in: 0, 000. Value of data_out: 0, 000.
# Value of i:          30. Value of address: 30, 11110. Value of data_in: 0, 000. Value of data_out: 0, 000.
# Value of i:          31. Value of address: 31, 11111. Value of data_in: 0, 000. Value of data_out: 0, 000.
# ----------------- write in numbers 0-7 -------------------
# Value of i:           0. Value of address:  0, 00000. Value of data_in: 0, 000. Value of data_out: 0, 000.
# Value of i:           1. Value of address:  1, 00001. Value of data_in: 1, 001. Value of data_out: 0, 000.
# Value of i:           2. Value of address:  2, 00010. Value of data_in: 2, 010. Value of data_out: 1, 001.
# Value of i:           3. Value of address:  3, 00011. Value of data_in: 3, 011. Value of data_out: 2, 010.
# Value of i:           4. Value of address:  4, 00100. Value of data_in: 4, 100. Value of data_out: 3, 011.
# Value of i:           5. Value of address:  5, 00101. Value of data_in: 5, 101. Value of data_out: 4, 100.
# Value of i:           6. Value of address:  6, 00110. Value of data_in: 6, 110. Value of data_out: 5, 101.
# Value of i:           7. Value of address:  7, 00111. Value of data_in: 7, 111. Value of data_out: 6, 110.
# Value of i:           8. Value of address:  8, 01000. Value of data_in: 0, 000. Value of data_out: 7, 111.
# Value of i:           9. Value of address:  9, 01001. Value of data_in: 1, 001. Value of data_out: 0, 000.
# Value of i:          10. Value of address: 10, 01010. Value of data_in: 2, 010. Value of data_out: 1, 001.
# Value of i:          11. Value of address: 11, 01011. Value of data_in: 3, 011. Value of data_out: 2, 010.
# Value of i:          12. Value of address: 12, 01100. Value of data_in: 4, 100. Value of data_out: 3, 011.
```

```
# ------------------ write enable OFF --------------------
# Value of i:           0. Value of address:  0, 00000. Value of data_in: 0, 000. Value of data_out: 7, 111.
# Value of i:           1. Value of address:  1, 00001. Value of data_in: 0, 000. Value of data_out: 0, 000.
# Value of i:           2. Value of address:  2, 00010. Value of data_in: 0, 000. Value of data_out: 1, 001.
# Value of i:           3. Value of address:  3, 00011. Value of data_in: 0, 000. Value of data_out: 2, 010.
# Value of i:           4. Value of address:  4, 00100. Value of data_in: 0, 000. Value of data_out: 3, 011.
# Value of i:           5. Value of address:  5, 00101. Value of data_in: 0, 000. Value of data_out: 4, 100.
# Value of i:           6. Value of address:  6, 00110. Value of data_in: 0, 000. Value of data_out: 5, 101.
# Value of i:           7. Value of address:  7, 00111. Value of data_in: 0, 000. Value of data_out: 6, 110.
# Value of i:           8. Value of address:  8, 01000. Value of data_in: 0, 000. Value of data_out: 7, 111.
# Value of i:           9. Value of address:  9, 01001. Value of data_in: 0, 000. Value of data_out: 0, 000.
# Value of i:          10. Value of address: 10, 01010. Value of data_in: 0, 000. Value of data_out: 1, 001.
# Value of i:          11. Value of address: 11, 01011. Value of data_in: 0, 000. Value of data_out: 2, 010.
```

Figure 3. Transcript for task2 module

Figure 3 shows the transcript for the task2 module, which is a SystemVerilog implementation of the memory module from task1. As seen in the transcript ouptuts, task2 is very similar to task1, and it has the same transcript output as task1 with the same 1 cycle clock delay. To be certain that the two modules had the same functionality, we included the same $display outputs in both modules, and they match.

Enabled Counter:

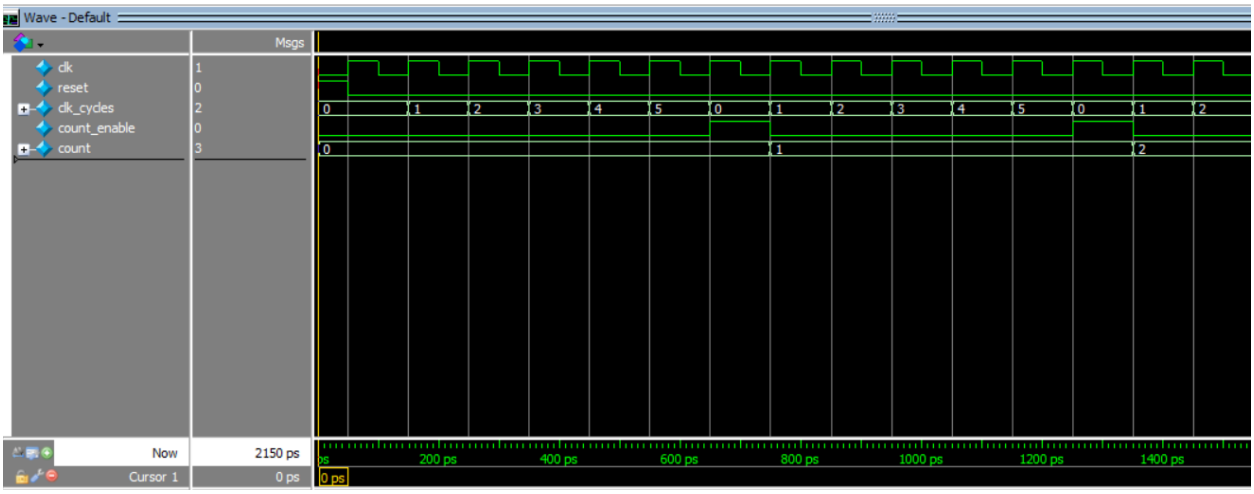


Figure 4. Waveforms for counter module

The enabled counter module in practice cycles through outputs of 0-31, incrementing the output value every second until it reaches 31, after which it returns to 0 and starts the cycle again. It does this by driving the count_enable variable HIGH every 50 million cycles (since the CLOCK_50 crystal oscillator oscillates at 50 million cycles per second). To testbench this module, we changed the count_enable to be driven HIGH every 5 cycles to testbench.

The next few figures show the waveforms and terminal outputs for task3 (top-level DE1_SoC). DE1_SoC instantiates the task2 and task3 modules, but it also passes all hardware inputs through a double flipflop first. This is to design against metastability, for example if the inputs were to change too close to the clock edges. First, we testbench the task2 functionality of the top-level module. This is when SW[9] is set to 0. We testbench in a similar way to the above two modules (tasks 1 and 2).

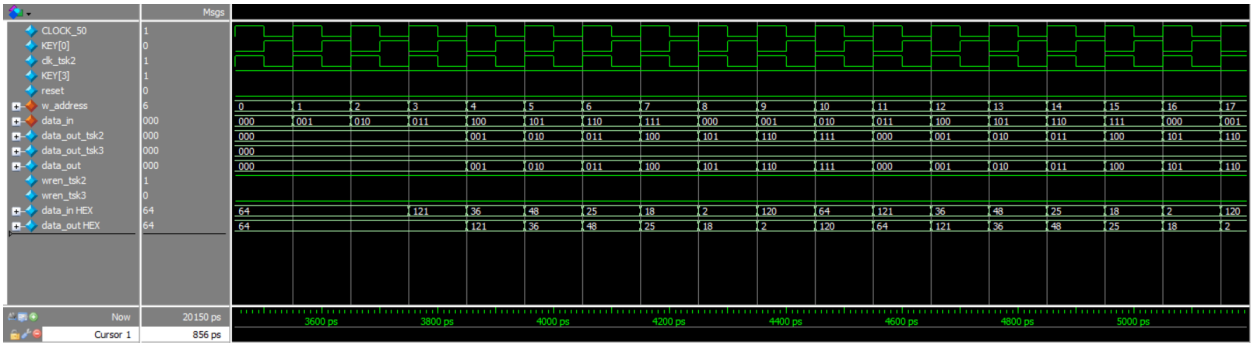


Figure 5. Task2, write enable ON

Figure 5 shows a section of the waveforms for the task2 functionality. It shows the write_address incrementing by 1 each time, and the data_in displays the least significant 3 bits of the write_address. The data_out output is seen to be following the data_in input, with a delay of 3 clock cycles. This is due to the same 1-cycle delay as before, plus the 2-cycle delay from the double flipflop input stabilizer module.

```
# ----------------- TESTBENCH TASK 2 -------------------
# ----------------- turn write enable ON -------------------
# Value of i:           0. Value of address:  0, 00000. Value of data_in: x, xxx. Value of data_out: x, xxx.
# Value of i:           1. Value of address:  1, 00001. Value of data_in: x, xxx. Value of data_out: x, xxx.
# Value of i:           2. Value of address:  2, 00010. Value of data_in: 0, 000. Value of data_out: x, xxx.
# Value of i:           3. Value of address:  3, 00011. Value of data_in: 0, 000. Value of data_out: 0, 000.
# Value of i:           4. Value of address:  4, 00100. Value of data_in: 0, 000. Value of data_out: 0, 000.
# Value of i:           5. Value of address:  5, 00101. Value of data_in: 0, 000. Value of data_out: 0, 000.
# Value of i:           6. Value of address:  6, 00110. Value of data_in: 0, 000. Value of data_out: 0, 000.
# Value of i:           7. Value of address:  7, 00111. Value of data_in: 0, 000. Value of data_out: 0, 000.
# Value of i:           8. Value of address:  8, 01000. Value of data_in: 0, 000. Value of data_out: 0, 000.
# Value of i:           9. Value of address:  9, 01001. Value of data_in: 0, 000. Value of data_out: 0, 000.
# Value of i:          10. Value of address: 10, 01010. Value of data_in: 0, 000. Value of data_out: 0, 000.
# Value of i:          11. Value of address: 11, 01011. Value of data_in: 0, 000. Value of data_out: 0, 000.
# Value of i:          12. Value of address: 12, 01100. Value of data_in: 0, 000. Value of data_out: 0, 000.
```

...

```
# Value of i:           0. Value of address:  0, 00000. Value of data_in: 0, 000. Value of data_out: 0, 000.
# Value of i:           1. Value of address:  1, 00001. Value of data_in: 0, 000. Value of data_out: 0, 000.
# Value of i:           2. Value of address:  2, 00010. Value of data_in: 0, 000. Value of data_out: 0, 000.
# Value of i:           3. Value of address:  3, 00011. Value of data_in: 1, 001. Value of data_out: 0, 000.
# Value of i:           4. Value of address:  4, 00100. Value of data_in: 2, 010. Value of data_out: 1, 001.
# Value of i:           5. Value of address:  5, 00101. Value of data_in: 3, 011. Value of data_out: 2, 010.
# Value of i:           6. Value of address:  6, 00110. Value of data_in: 4, 100. Value of data_out: 3, 011.
# Value of i:           7. Value of address:  7, 00111. Value of data_in: 5, 101. Value of data_out: 4, 100.
# Value of i:           8. Value of address:  8, 01000. Value of data_in: 6, 110. Value of data_out: 5, 101.
# Value of i:           9. Value of address:  9, 01001. Value of data_in: 7, 111. Value of data_out: 6, 110.
# Value of i:          10. Value of address: 10, 01010. Value of data_in: 0, 000. Value of data_out: 7, 111.
# Value of i:          11. Value of address: 11, 01011. Value of data_in: 1, 001. Value of data_out: 0, 000.
# Value of i:          12. Value of address: 12, 01100. Value of data_in: 2, 010. Value of data_out: 1, 001.
```

Figure 6. Terminal output of task 2, write enable ON then OFF

The same three-clock cycle delay is seen in Figure 6, which shows the terminal output of the task2 testbench when write enable is ON.

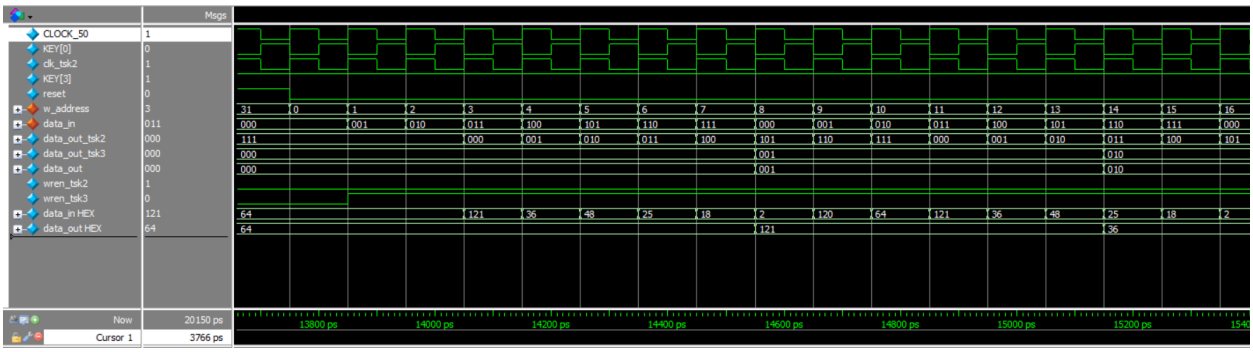
Now, we turn off write enable:

```
# ----------------- turn write enable OFF -------------------
# Value of i:           0. Value of address:  0, 00000. Value of data_in: 7, 111. Value of data_out: 6, 110.
# Value of i:           1. Value of address:  1, 00001. Value of data_in: 7, 111. Value of data_out: 7, 111.
# Value of i:           2. Value of address:  2, 00010. Value of data_in: 0, 000. Value of data_out: 7, 111.
# Value of i:           3. Value of address:  3, 00011. Value of data_in: 0, 000. Value of data_out: 0, 000.
# Value of i:           4. Value of address:  4, 00100. Value of data_in: 0, 000. Value of data_out: 1, 001.
# Value of i:           5. Value of address:  5, 00101. Value of data_in: 0, 000. Value of data_out: 2, 010.
# Value of i:           6. Value of address:  6, 00110. Value of data_in: 0, 000. Value of data_out: 3, 011.
# Value of i:           7. Value of address:  7, 00111. Value of data_in: 0, 000. Value of data_out: 4, 100.
# Value of i:           8. Value of address:  8, 01000. Value of data_in: 0, 000. Value of data_out: 5, 101.
# Value of i:           9. Value of address:  9, 01001. Value of data_in: 0, 000. Value of data_out: 6, 110.
# Value of i:          10. Value of address: 10, 01010. Value of data_in: 0, 000. Value of data_out: 7, 111.
```

Figure 7. Write enable turned OFF

As seen in Figure 7, when we turn off write enable, the value stored inside each address does not change from what was previously stored, and the new value of data_in does not replace the old data. Thus, the data_out outputs what was previously stored in the memory block, and we cannot write to the array again until write enable is turned on again.

Then, we testbench the dual-port RAM block in the same way as the single-port RAM block. First, we toggle SW[9] ON, switching the functionality from task2 to task3. In this testbench, however, we start by leaving write_enable turned off to see the values stored (by default) in the Quartus dual-port IP memory block. Then, starting with write-enable on, we write data to the module and read it, and then turn write-enable off and try to write to the module again. Again, we see the same result, as shown in Figure 8.




```
# ----------------- NOW TESTBENCH TASK 3 -------------------
# ----------------- data_out should show the same value for 6 cycles at a time -------------------
# ----------------- turn write enable OFF to see .mif output -------------------
# Value of i:           0. Value of address (w & r):  0, 31. Value of data_in: 0, 000. Value of data_out: 7, 111.
# Value of i:           1. Value of address (w & r):  1, 31. Value of data_in: 0, 000. Value of data_out: 7, 111.
# Value of i:           2. Value of address (w & r):  2,  0. Value of data_in: 0, 000. Value of data_out: 0, 000.
# Value of i:           3. Value of address (w & r):  3,  0. Value of data_in: 0, 000. Value of data_out: 0, 000.
# Value of i:           4. Value of address (w & r):  4,  0. Value of data_in: 0, 000. Value of data_out: 0, 000.
# Value of i:           5. Value of address (w & r):  5,  0. Value of data_in: 0, 000. Value of data_out: 0, 000.
# Value of i:           6. Value of address (w & r):  6,  0. Value of data_in: 0, 000. Value of data_out: 0, 000.
# Value of i:           7. Value of address (w & r):  7,  1. Value of data_in: 0, 000. Value of data_out: 0, 000.
# Value of i:           8. Value of address (w & r):  8,  1. Value of data_in: 0, 000. Value of data_out: 0, 000.
# Value of i:           9. Value of address (w & r):  9,  1. Value of data_in: 0, 000. Value of data_out: 0, 000.
# Value of i:          10. Value of address (w & r): 10,  1. Value of data_in: 0, 000. Value of data_out: 0, 000.
# Value of i:          11. Value of address (w & r): 11,  1. Value of data_in: 0, 000. Value of data_out: 0, 000.
# Value of i:          12. Value of address (w & r): 12,  1. Value of data_in: 0, 000. Value of data_out: 0, 000.
```

...

Write enable ON:

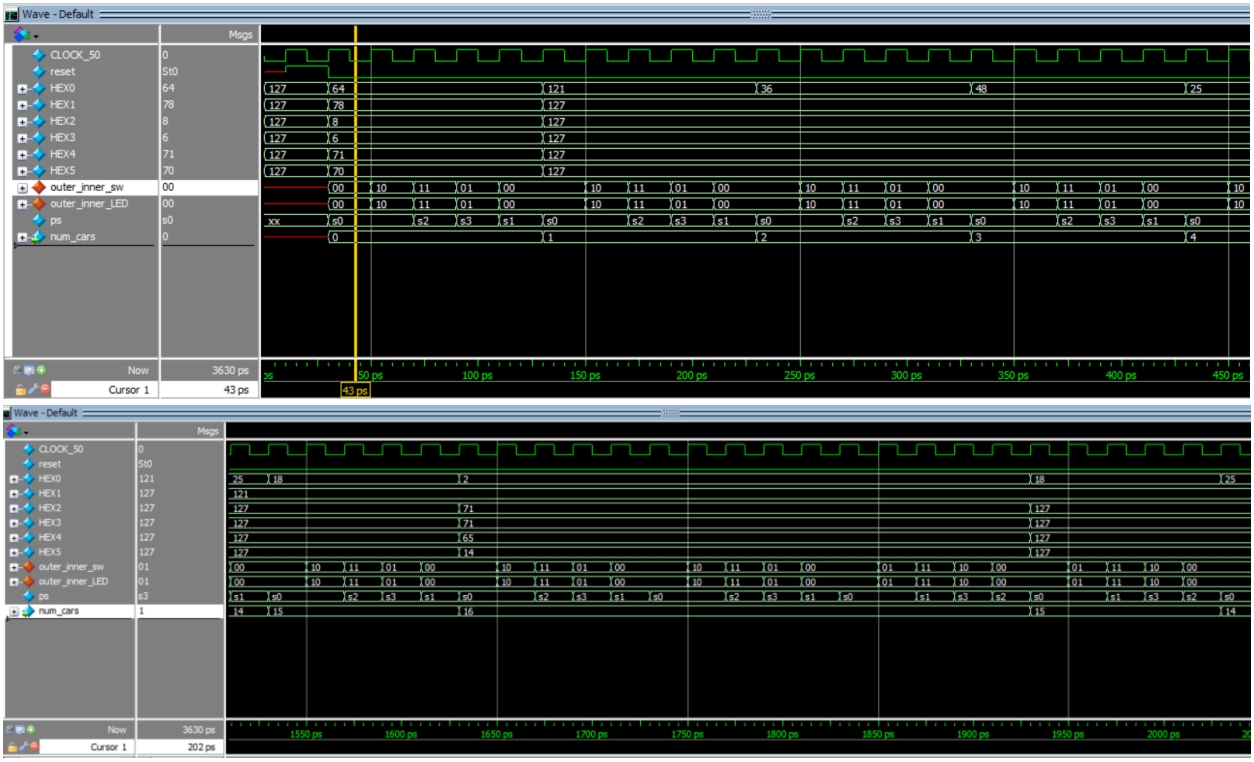
```
# Value of i:           0. Value of address (w & r):  0,  0. Value of data_in: 0, 000. Value of data_out: 0, 000.
# Value of i:           1. Value of address (w & r):  1,  0. Value of data_in: 0, 000. Value of data_out: 0, 000.
# Value of i:           2. Value of address (w & r):  2,  0. Value of data_in: 0, 000. Value of data_out: 0, 000.
# Value of i:           3. Value of address (w & r):  3,  0. Value of data_in: 1, 001. Value of data_out: 0, 000.
# Value of i:           4. Value of address (w & r):  4,  0. Value of data_in: 2, 010. Value of data_out: 0, 000.
# Value of i:           5. Value of address (w & r):  5,  0. Value of data_in: 3, 011. Value of data_out: 0, 000.
# Value of i:           6. Value of address (w & r):  6,  0. Value of data_in: 4, 100. Value of data_out: 0, 000.
# Value of i:           7. Value of address (w & r):  7,  1. Value of data_in: 5, 101. Value of data_out: 0, 000.
# Value of i:           8. Value of address (w & r):  8,  1. Value of data_in: 6, 110. Value of data_out: 1, 001.
# Value of i:           9. Value of address (w & r):  9,  1. Value of data_in: 7, 111. Value of data_out: 1, 001.
# Value of i:          10. Value of address (w & r): 10,  1. Value of data_in: 0, 000. Value of data_out: 1, 001.
# Value of i:          11. Value of address (w & r): 11,  1. Value of data_in: 1, 001. Value of data_out: 1, 001.
# Value of i:          12. Value of address (w & r): 12,  1. Value of data_in: 2, 010. Value of data_out: 1, 001.
# Value of i:          13. Value of address (w & r): 13,  2. Value of data_in: 3, 011. Value of data_out: 1, 001.
# Value of i:          14. Value of address (w & r): 14,  2. Value of data_in: 4, 100. Value of data_out: 2, 010.
# Value of i:          15. Value of address (w & r): 15,  2. Value of data_in: 5, 101. Value of data_out: 2, 010.
# Value of i:          16. Value of address (w & r): 16,  2. Value of data_in: 6, 110. Value of data_out: 2, 010.
# Value of i:          17. Value of address (w & r): 17,  2. Value of data_in: 7, 111. Value of data_out: 2, 010.
# Value of i:          18. Value of address (w & r): 18,  2. Value of data_in: 0, 000. Value of data_out: 2, 010.
# Value of i:          19. Value of address (w & r): 19,  3. Value of data_in: 1, 001. Value of data_out: 2, 010.
# Value of i:          20. Value of address (w & r): 20,  3. Value of data_in: 2, 010. Value of data_out: 3, 011.
# Value of i:          21. Value of address (w & r): 21,  3. Value of data_in: 3, 011. Value of data_out: 3, 011.
# Value of i:          22. Value of address (w & r): 22,  3. Value of data_in: 4, 100. Value of data_out: 3, 011.
```

```
# ----------------- turn write enable OFF -------------------
# ----------------- outputs data written in in the past for-loop -------------------
# Value of i:          0. Value of address (w & r):  0,  5. Value of data_in: 6, 110. Value of data_out: 5, 101.
# Value of i:          1. Value of address (w & r):  1,  5. Value of data_in: 7, 111. Value of data_out: 5, 101.
# Value of i:          2. Value of address (w & r):  2,  5. Value of data_in: 0, 000. Value of data_out: 5, 101.
# Value of i:          3. Value of address (w & r):  3,  5. Value of data_in: 0, 000. Value of data_out: 5, 101.
# Value of i:          4. Value of address (w & r):  4,  5. Value of data_in: 0, 000. Value of data_out: 5, 101.
# Value of i:          5. Value of address (w & r):  5,  6. Value of data_in: 0, 000. Value of data_out: 5, 101.
# Value of i:          6. Value of address (w & r):  6,  6. Value of data_in: 0, 000. Value of data_out: 6, 110.
# Value of i:          7. Value of address (w & r):  7,  6. Value of data_in: 0, 000. Value of data_out: 6, 110.
# Value of i:          8. Value of address (w & r):  8,  6. Value of data_in: 0, 000. Value of data_out: 6, 110.
# Value of i:          9. Value of address (w & r):  9,  6. Value of data_in: 0, 000. Value of data_out: 6, 110.
# Value of i:         10. Value of address (w & r): 10,  6. Value of data_in: 0, 000. Value of data_out: 6, 110.
# Value of i:         11. Value of address (w & r): 11,  7. Value of data_in: 0, 000. Value of data_out: 6, 110.
# Value of i:         12. Value of address (w & r): 12,  7. Value of data_in: 0, 000. Value of data_out: 7, 111.
# Value of i:         13. Value of address (w & r): 13,  7. Value of data_in: 0, 000. Value of data_out: 7, 111.
# Value of i:         14. Value of address (w & r): 14,  7. Value of data_in: 0, 000. Value of data_out: 7, 111.
# Value of i:         15. Value of address (w & r): 15,  7. Value of data_in: 0, 000. Value of data_out: 7, 111.
```

Figure 8. Waveforms & transcript for task3 module (DE1_SoC)

The dual-port Quartus IP memory block works in the same way as the single-port block, except that there are now separate read and write addresses.

Finally, we testbench the HEX displays, which call the seg7 module. Each HEX displays either the time (in seconds, from the counter module testbenched in Figure 4), the write or read address, the data_in input as a decimal, or the data_out output as a decimal. The results of this testbenching are shown in Figures 9 and 10.



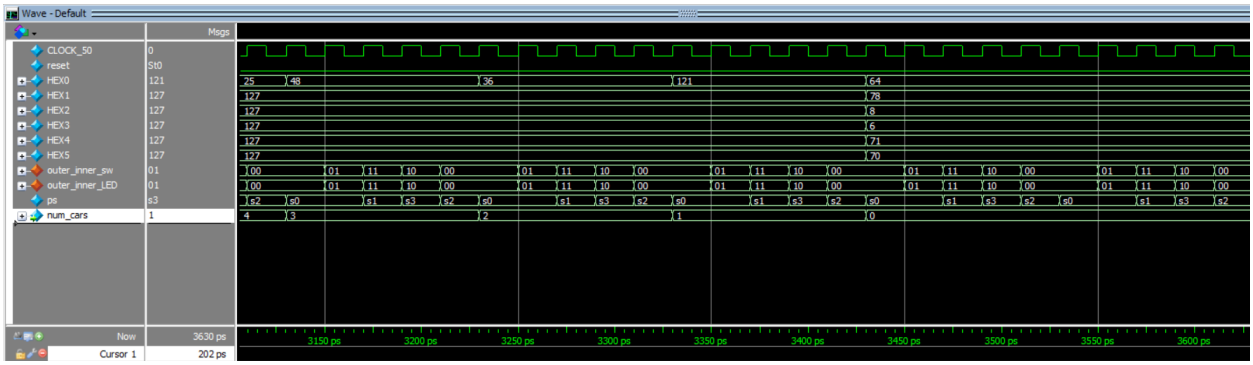


Figure 9. Counter increments and decrements, allowing the HEX values to change

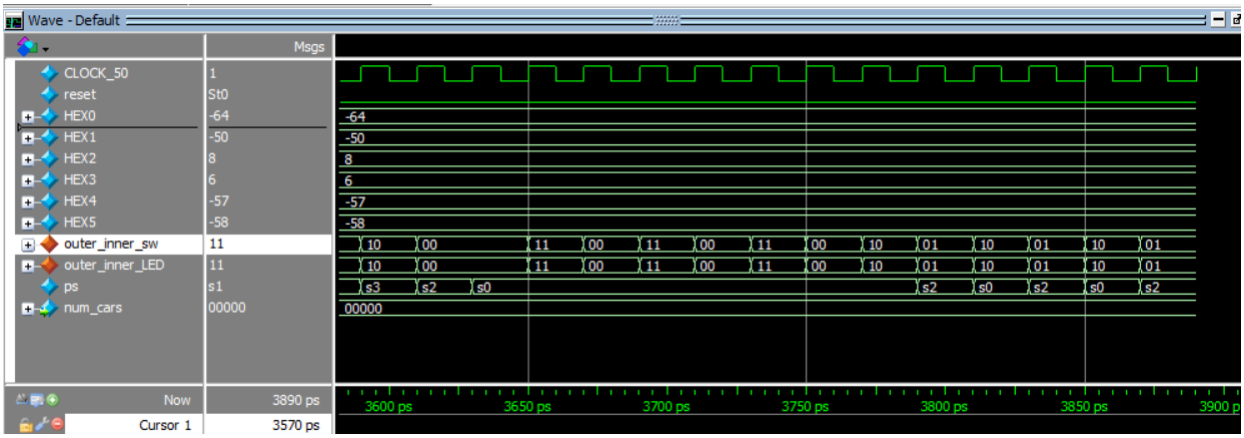


Figure 10. Waveforms showing that the HEX does not increment with random combinations of the sensor inputs

The full testbenching of all modules, and each aspect of each module, allowed us to determine that our system was working as intended. We then uploaded the code to LabsLand and simulated it on the virtual FPGA, and it worked there as intended as well.

Experience Report

This lab was easy to understand, but when implementing our initial designs, we ran into issues with our logic. The most difficult and time-consuming part was the debugging process, even as we testbenched each module individually modularly before integrating everything within the top-level module. Writing the testbenches and figuring out what to testbench also took a bulk of our time, but ultimately we figured out how to testbench to show the signals we wanted to see, and were then able to upload the code to the virtual hardware almost seamlessly.

The lab took approximately 15 hours, broken down as follows:

- Reading and interpreting lab specs for task #1 and #2 – 1 hour
- Following the instructions for task #1 and #2 – 3 hours
- Debugging task #2 – 1 hour

- Reading and interpreting lab specs for task #3 – 0.25 hours
- Implementing initial version of task #3 – 1 hour
- Debugging initial version of task #3 – 2 hours
- Implementing second version of task #3 – 2.5 hours
- Debugging waveform outputs – 0.75 hours
- Changed implementation of task #3 again after re-reading spec – 1.5 hours
- Debug waveforms again – 0.25 hours
- Upload files to LabsLand to test – 0.5 hours
- Writing report – 2 hours (but concurrently so it didn't really add too much extra time)